

Utime Team Manager

Plan de proyecto – Unidad de Flutter

Aplicacion tipo FC Ultimate Team en Flutter / Dart

Integrantes	Sebastian Ramirez • Joaquin Parraud
Repositorio	Utime-Team-Manager (GitHub)
Stack	Flutter, Dart, Riverpod, go_router, Hive, Supabase (futuro)
Fecha de inicio	Lunes 13 de julio (Clase 2)
Entrega / presentacion	Semana del 20 al 25 de julio

Índice

1. Resumen y contexto	3
2. Objetivos y alcance	3
3. Stack tecnologico	4
4. Roles y responsabilidades	4
5. Hitos del proyecto	4
6. Cronograma (Gantt)	4
7. Desglose de tareas por sprint	5
7.1. Sprint 1 – Login y base (13–19 jul)	5
7.2. Sprint 2 – Datos y juego (20–25 jul)	5
8. Estrategia de ramas Git	6
9. Riesgos y mitigaciones	6
10. Definición de Hecho (DoD) y rubrica	6

1. Resumen y contexto

Ultime Team Manager es una aplicacion movil inspirada en *FC Ultimate Team*. El usuario inicia sesion, arma su equipo de once colocando cartas de futbolistas en sus posiciones (formacion **4-3-3** por defecto), compra y vende cartas en un mercado, simula partidos cuyo resultado depende de la valoracion media de cada equipo y compite en una liga que reparte monedas segun el resultado.

El proyecto se construye sobre la app base de la Unidad 1 (un contador con `setState`) y evoluciona hacia una arquitectura por capas. El desarrollo es **offline-first** pero disenado para escalar a un backend online (Supabase) sin reescribir la capa de presentacion.

Meta de la unidad: llevar la idea a una app funcional en tres clases y presentarla la semana del 20 al 25 de julio, cumpliendo los requisitos minimos (varias pantallas + navegacion, una lista con CRUD, conexion a datos, UI cuidada y repositorio en GitHub con ramas por tarea).

2. Objetivos y alcance

Objetivo general

Entregar una app Flutter funcional y presentable que implemente el ciclo completo: *login* → *armar plantilla* → *mercado* → *simular partido* → *liga y monedas*.

Dentro del alcance

- Login con validacion de campos y sesion persistente (offline).
- Plantilla de 11 jugadores en formacion 4-3-3 sobre un campo.
- Cartas con valoracion y atributos; datos desde JSON semilla y API publica.
- Mercado con CRUD (comprar/vender) sobre la coleccion del usuario.
- Simulacion de partidos por valoracion media y liga con reparto de monedas.
- Estados de carga y de vacio en todas las listas.

Fuera del alcance (por ahora)

- Backend online real (queda la arquitectura lista para Supabase).
- Multijugador en tiempo real, notificaciones push y micropagos.
- Motor de simulacion minuto a minuto (se usa un modelo probabilistico simple).

3. Stack tecnologico

Area	Eleccion
Framework / lenguaje	Flutter, Dart (sdk ^3.12.2)
Gestion de estado	Riverpod (flutter_riverpod)
Navegacion	go_router
Persistencia local	Hive (cartas, plantilla, liga) + shared_preferences (sesion)
Backend online (futuro)	Supabase (Auth + Postgres)
Origen de cartas	JSON semilla en assets/data + API publica de futbol (TheSportsDB)
Testing / Linting	flutter_test + flutter_lints
Control de versiones	Git + GitHub (ramas por tarea, pull requests)

4. Roles y responsabilidades

Dos integrantes con areas principales; el trabajo se reparte por *feature* y se integra mediante pull requests para asegurar commits de ambos.

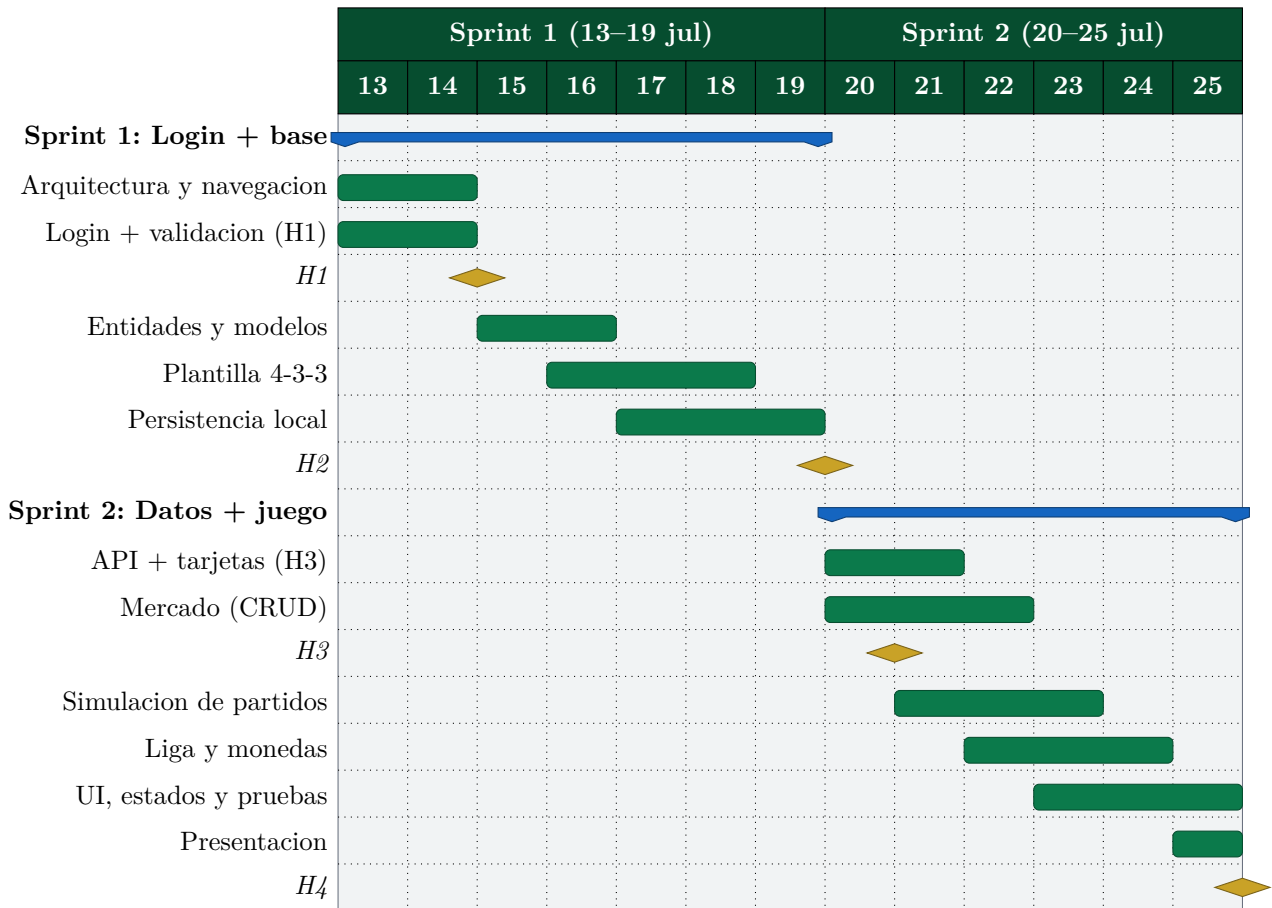
Integrante	Responsabilidades
Sebastian Ramirez	Arquitectura base y navegacion, login + validacion, sesion, entidades del dominio, pantalla de plantilla (4-3-3), tema/UI.
Joaquin Parraud	Mercado (CRUD), consumo de API y modelos de datos, simulacion de partidos, liga y economia de monedas, estados de carga/vacio.

5. Hitos del proyecto

Fecha	Hito	Entregable
Lun 06-07	H0 – Arranque	Grupo, idea definida y repositorio creado (hecho).
Lun 13-07	H1 – Login con validacion	Login funcional con validacion, sesion y navegacion entre 3+ pantallas.
Dom 19-07	H2 – Plantilla y datos	Modelos, plantilla 4-3-3, persistencia local y primeras cartas.
Lun 20-07	H3 – API + tarjetas	Consumo de API mostrado en tarjetas dinamicas; mercado con CRUD (cierre de proyecto).
20-25 jul	H4 – Presentacion	Simulacion, liga, monedas, UI pulida, pruebas y presentacion grupal.

6. Cronograma (Gantt)

Del 13 al 25 de julio, organizado en dos sprints alineados con las clases.



Los numeros del eje corresponden a los dias de julio. Los rombos dorados marcan los hitos H1–H4.

7. Desglose de tareas por sprint

7.1 Sprint 1 – Login y base (13–19 jul)

ID	Tarea	Responsable	RF
T1.1	Configurar arquitectura por capas y <code>go_router</code>	Sebastian	—
T1.2	Renombrar proyecto y anadir dependencias (Riverpod, Hive)	Sebastian	—
T1.3	Pantalla de login con validacion de campos	Sebastian	RF1
T1.4	Sesion persistente con <code>shared_preferences</code>	Sebastian	RF1
T1.5	Entidades del dominio (Player, Card, Team, Match, League)	Joaquin	RF3
T1.6	Semilla JSON de jugadores en <code>assets/data</code>	Joaquin	RF3
T1.7	Pantalla de plantilla con formacion 4-3-3	Sebastian	RF2
T1.8	Persistencia local de la plantilla (Hive)	Joaquin	RF2

7.2 Sprint 2 – Datos y juego (20–25 jul)

ID	Tarea	Responsable	RF
T2.1	Consumir API publica de futbol y mapear a modelos	Joaquin	RF3
T2.2	Mostrar cartas dinamicas con estados de carga/vacio	Joaquin	RF3
T2.3	Mercado: comprar y vender (CRUD completo)	Joaquin	RF4
T2.4	Motor de simulacion por valoracion media	Sebastian	RF5
T2.5	Liga: calendario, clasificacion y reparto de monedas	Joaquin	RF6
T2.6	Dashboard de inicio (monedas y accesos)	Sebastian	—
T2.7	Pulido de UI, tema y widgets reutilizables	Sebastian	—
T2.8	Pruebas de widgets/logica y preparacion de la demo	Ambos	—

8. Estrategia de ramas Git

- **main** – rama estable y presentable.
- **develop** – integracion continua del trabajo del equipo.
- **feature/<tarea>** – una rama por tarea (p.ej. **feature/login-validacion**).
- Cada *feature* se integra mediante **pull request** revisado por el otro integrante.

Objetivo: historial claro con **commits de ambos integrantes** y trazabilidad tarea → rama → PR.

9. Riesgos y mitigaciones

Riesgo	Impacto	Mitigacion
Plazo muy ajustado (2 semanas)	Alto	Priorizar RF minimos; dejar simulacion/liga como incremento final.
API externa caida o con limites	Medio	Caer a la semilla JSON local (offline-first).
Conflictos de merge	Medio	Ramas por tarea y PRs pequenos y frecuentes.
Alcance excesivo (scope creep)	Medio	Congelar el alcance tras H3; extras solo si sobra tiempo.
Curva de Riverpod/Hive	Bajo	Repartir por area y apoyarse en ejemplos de clase.

10. Definicion de Hecho (DoD) y rubrica

Una tarea esta **Hecha** cuando: compila sin errores, pasa el *linter*, tiene los estados de carga/vacio cuando aplica, y esta integrada en **main** via PR revisado.

Mapeo con los requisitos minimos de la unidad:

Requisito de la unidad	Como se cumple
Varias pantallas + navegacion	Login, Home, Perfil (go_router).
Una lista con CRUD	Mercado y plantilla
Conexion a datos	API publica de futbol
UI cuidada con widgets	Layouts, cartas, cards
Repositorio en GitHub	Ramas por tarea, Pull Request